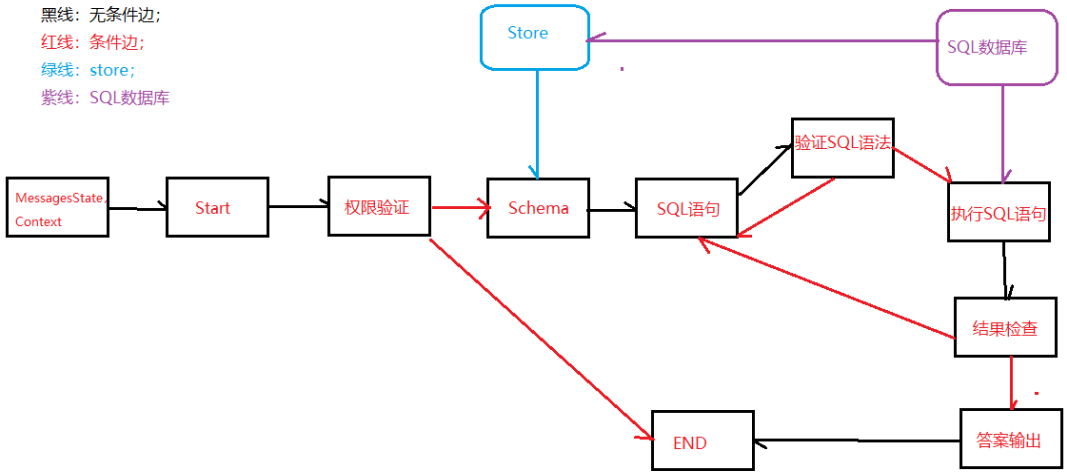
2026.11.06langgraph 作业

作业：根据下面的图片，写出一个SQL查询的langgraph系统

1. 示意图：



2. graph结构分析图



3. 代码

```
from typing import TypedDict
from langchain_core.messages import HumanMessage, AIMessage, AnyMessage,
SystemMessage
from langgraph.store.memory import InMemoryStore
from langgraph.graph import StateGraph, START, END, MessagesState
from langgraph.runtime import Runtime
from langchain_openai import ChatOpenAI
from langchain_openai import OpenAIEmbeddings
import sqlglot
import pymysql
import json



#====================================1.基础配置==============================
#1.构建模型
model = ChatOpenAI(model="qwen3.8-max",
                   base_url="https://cn-
beijing.maas.qianwenaiapi.com/compatible-mode/v1",
```

```
                    api_key="sk-ws-
H.YLHYYPX.Yrak.MEUCIQCTdzglRMGy37HmU1VnYcSH6Hyce71ViLrj2zqZP2g3DAIgTwT-
lgL83capmx74qqUNUreEATzY-UoeibaY1bRduV0",
                    temperature=0)


# 2.数据库连接配置
DB_CONFIG = {
    "host": "192.168.1.103",
    "port": 3306,
    "user": "root",
    "password": "zb19891225",
    "database": "lang memory", # 你的数据库名
    "charset": "utf8mb4",
    "cursorclass": pymysql.cursors.DictCursor # 返回字典格式，方便后续处理
}

def get_db_connection():
    return pymysql.connect(**DB_CONFIG)

# ========================================
# 3. 构建store
class Context(TypedDict):
    user_id: str

store = InMemoryStore(
    index={
        "embed": OpenAIEmbeddings(model="qwen3.7-text-embedding-
flash",base_url="https://cn-beijing.maas.qianwenaiapi.com/compatible-
mode/v1",
                                  api_key="sk-ws-
H.YIPXDYD.GQ3w.MEQCIGUk6xKey1vwED7Q_f8nIOERsDRWSfsbLZw89wKsQf4rAiA-
f9DPuXKRa1copGfbyLvFktFzwtZgg4q7YVlhUtA3iQ",
                                  check_embedding_ctx_length=False),
        "dims": 1536, }
        )

namespace = ("memories", "user_1")

def init_store():
    employees_schema = """
    表名: employees (员工表)
    字段:
    - emp_no: int, 员工编号 (主键)
    - birth_date: date, 出生日期
    - first_name: varchar(14), 名
    - last_name: varchar(16), 姓
    - gender: enum, 性别 (M/F)
    - hire_date: date, 入职日期
    """

    dept_manager_schema = """
    表名: dept_manager (部门经理表)
    字段:
    - emp_no: int, 员工编号
    - dept_no: char(4), 部门编号
    - from_date: date, 任职开始日期
    - to_date: date, 任职结束日期
```

```
    """

    store.put(namespace, "employees", {"content": employees_schema,
"table_name": "employees"})
    store.put(namespace, "dept_manager", {"content": dept_manager_schema,
"table_name": "dept_manager"})

init_store()

# ====================================2. 构建图纸
==================================
# 1.节点定义

#1.1 权限节点
def check_permission(state: MessagesState, runtime: Runtime[Context]):
    user_id = runtime.context.get("user_id")
    if user_id not in ["admin_001", "user_123"]:
        return {"messages": [AIMessage(content=f"权限不足，用户 {user_id} 无权访
问。", name="error")]}
    return {} # 权限通过，什么都不用返回

#1.2 schema节点
def retrieve_schema(state: MessagesState):
    # 获取用户的问题
    user_question = state["messages"][0].content

    # 去 Store 中做相似度检索 (limit=2 表示最多取两张表)
    results = store.search(namespace, query=user_question, limit=2)

    # 拼接检索到的 Schema 文本
    schema_contents = []
    for item in results:
        schema_contents.append(item.value["content"])

    # 如果没检索到（兜底），就给默认的
    if not schema_contents:
        final_schema = "未找到相关表结构"
    else:
        final_schema = "\n\n".join(schema_contents)

    return {"messages": [AIMessage(content=final_schema,
name="schema_retriever")]}

#1.3 输出SQL语法节点
def generate_sql(state: MessagesState):
    user_question = state["messages"][0].content

    schema_context = state["messages"][-1].content

    system_prompt = """你是一个精通 MySQL 的资深数据库专家。
你的任务是根据给定的【数据库Schema】和【用户问题】，生成一条可以正确执行的 SQL 语句。

要求：
1. 只能使用 Schema 中存在的表和字段。
2. 只输出纯 SQL 语句，不要包含任何 markdown 标记（如 ```sql ），不要输出任何解释性文
字。
3. 如果你使用了多表 JOIN 查询，请务必使用 DISTINCT 防止数据重复放大。
4. 日期条件请严格按照字面意思翻译，例如“以前”通常使用 `< 'YYYY-01-01'`，不要过度联想。
```

```
【数据库Schema】
{schema}
""".format(schema=schema_context)

    # 构造 Human Prompt
    human_prompt = f"用户问题\n{user_question}"

    # 调用大模型
    response = model.invoke([
        SystemMessage(content=system_prompt),
        HumanMessage(content=human_prompt)
    ])

    # 提取并清洗大模型的输出 (防止有些模型不听话加上了 ```sql )
    raw_sql = response.content.strip()
    if raw_sql.startswith("```"):
        # 去除首尾的 markdown 标记
        raw_sql = raw_sql.replace("```sql", "").replace("```", "").strip()

    # 将 SQL 塞进 AIMessage 返回，等待下一个节点校验
    print(f"生成的 SQL: {raw_sql}") # <--- 加这一行
    return {"messages": [AIMessage(content=raw_sql, name="sql_generator")]}

#1.4 SQL语法检查节点
def validate_sql(state: MessagesState):
    sql = state["messages"][-1].content

    try:
        sqlglot.parse_one(sql, dialect="mysql")
        return {} # 校验通过，不追加消息
    except Exception as e:
        # 语法错误，追加一条报错消息
        return {"messages": [AIMessage(content=f"SQL 语法错误: {e}",
name="error")]}

#1.5 执行SQL节点
def execute_sql(state: MessagesState):
    sql = state["messages"][-1].content
    print(f"执行SQL: {sql}")                #数据库的语句
    conn = None
    try:
        conn = get_db_connection()
        with conn.cursor() as cursor:
            # 执行真实查询
            cursor.execute(sql)

            results = cursor.fetchall()

        # 将结果塞进 additional_kwargs
        return {"messages": [AIMessage(
            content="执行成功",
            name="executor",
            additional_kwargs={"data": results}
        )]}

    except pymysql.MySQLError as e:
        print(f"   [执行报错] {e}")
```

```
        # 返回错误消息，交给条件边处理
        return {"messages": [AIMessage(content=f"数据库执行失败: {e}",
name="error")]}
    finally:
        if conn:
            conn.close()

#1.6 结果检查节点
def check_result(state: MessagesState):
    last_msg = state["messages"][-1]
    data = last_msg.additional_kwargs.get("data")

    if not data:
        return {"messages": [AIMessage(content="抱歉，没有查到符合条件的数据。",
name="error")]}
    return {}

#1.7 答案回复节点
def generate_answer(state: MessagesState):

    user_question = state["messages"][0].content

    # 2. 安全地提取执行结果（遍历消息，找到 name="executor" 的）
    query_data = []
    for msg in reversed(state["messages"]):
        if msg.name == "executor":
            query_data = msg.additional_kwargs.get("data", [])
            break

    # 3. 将结构化数据转为 JSON 字符串
    data_str = json.dumps(query_data, ensure_ascii=False, indent=2)

    # 4. 构造 Prompt
    system_prompt = """你是一个专业的数据分析助手。
请根据用户的问题和数据库查询返回的数据结果，用自然、流畅的中文总结出最终答案。
要求：
1. 必须严格基于提供的数据进行回答，绝对不能凭空捏造数据。
2. 如果数据结果是空列表 []，请告诉用户“未查询到符合条件的数据”。
3. 回答需简明扼要，直接回应用户的问题。"""

    human_prompt = f"""【用户问题】
{user_question}

【数据库查询结果 (JSON格式)】
{data_str}
"""

    # 5. 调用大模型
    response = model.invoke([
        SystemMessage(content=system_prompt),
        HumanMessage(content=human_prompt)
    ])

    print( response.content)
    return {"messages": [AIMessage(content=response.content,
name="final_answer")]}
```

```python
#2.构建边

# 2.1. 条件边 (依据最新一条消息的 name 判断)
def route_permission(state: MessagesState):
    if state["messages"][-1].name == "error": return END
    return "retrieve_schema"

def route_validate_sql(state: MessagesState):
    if state["messages"][-1].name == "error": return "generate_sql"
    return "execute_sql"

def route_check_result(state: MessagesState):
    if state["messages"][-1].name == "error": return "generate_sql"
    return "generate_answer"


# 3. 构建图
builder = StateGraph(MessagesState, context_schema=Context)

builder.add_node("check_permission", check_permission)
builder.add_node("retrieve_schema", retrieve_schema)
builder.add_node("generate_sql", generate_sql)
builder.add_node("validate_sql", validate_sql)
builder.add_node("execute_sql", execute_sql)
builder.add_node("check_result", check_result)
builder.add_node("generate_answer", generate_answer)

builder.add_edge(START, "check_permission")
builder.add_conditional_edges("check_permission", route_permission,
["retrieve_schema", END])
builder.add_edge("retrieve_schema", "generate_sql")
builder.add_edge("generate_sql", "validate_sql")
builder.add_conditional_edges("validate_sql", route_validate_sql,
["execute_sql", "generate_sql"])
builder.add_edge("execute_sql", "check_result")
builder.add_conditional_edges("check_result", route_check_result,
["generate_answer","generate_sql"])
builder.add_edge("generate_answer", END)

app = builder.compile()


if __name__ == "__main__":
    inputs = {"messages": [HumanMessage(content="1987年以前出生的员工人数有多
少")]}
    context = {"user_id": "user_123"}

    app.invoke(inputs, context=context)
```

```
生成的 SQL: SELECT COUNT(*) FROM employees WHERE birth_date < '1987-01-01';
执行SQL: SELECT COUNT(*) FROM employees WHERE birth_date < '1987-01-01';
1987年以前出生的员工人数为 **300024** 人。
```